

Lisp machines are general-purpose computers designed to efficiently run Lisp as their main software and programming language, usually via hardware support. They are an example of a high-level language computer architecture. In a sense, they were the first commercial single-user workstations. Despite being modest in number (perhaps 7,000 units total as of 1988) Lisp machines commercially pioneered some now-commonplace technologies, including networking innovations such as Chaosnet, and effective garbage collection. Several firms built and sold Lisp machines in the 1980s: Symbolics (3600, 3640, XL1200, MacIvory, and other models), Lisp Machines Incorporated (LMI Lambda), Texas Instruments (Explorer, MicroExplorer), and Xerox (Interlisp-D workstations). The operating systems were written in Lisp Machine Lisp, Interlisp (Xerox), and later partly in Common Lisp.

History

Historical context

Artificial intelligence (AI) computer programs of the 1960s and 1970s intrinsically required what was then considered a huge amount of computer power, as measured in processor time and memory space. The power requirements of AI research were exacerbated by the Lisp symbolic programming language, when commercial hardware was designed and optimized for assembly- and Fortran-like programming languages. At first, the cost of such computer hardware meant that it had to be shared among many users. As integrated circuit technology shrank the size and cost of computers in the 1960s and early 1970s, and the memory needs of AI programs began to exceed the address space of the most common research computer, the Digital Equipment Corporation (DEC) PDP-10, researchers considered a new approach: a computer designed specifically to develop and run large artificial intelligence programs, and tailored to the semantics of the Lisp language. To provide consistent performance for interactive programs, these machines would often not be shared, but would be dedicated to a single user at a time.

Initial development

In 1973, Richard Greenblatt and Thomas Knight, programmers at Massachusetts Institute of Technology (MIT) Artificial Intelligence Laboratory (AI Lab), began what would become the MIT Lisp Machine Project when they first began building a computer hardwired to run certain basic Lisp operations, rather than run them in software, in a 24-bit tagged architecture. The machine also did incremental (or Arena) garbage collection. More specifically, since Lisp variables are typed at runtime rather than compile time, a simple addition of two variables could take five times as long on conventional hardware, due to test and branch instructions. Lisp Machines ran the tests in parallel with the more conventional single instruction additions. If the simultaneous tests failed, then the result was discarded and recomputed; this meant in many cases a speed increase by several factors. This simultaneous checking approach was used as well in testing the bounds of arrays when referenced, and other memory management necessities (not merely garbage collection or arrays).

Type checking was further improved and automated when the conventional byte word of 32 bits was lengthened to 36 bits for Symbolics 3600-model Lisp machines and eventually to 40 bits or more (usually, the excess bits not accounted for by the following were used for error-correcting codes). The first group of extra bits were used to hold type data, making the machine a tagged architecture, and the remaining bits were used to implement compressed data representation (CDR) coding (wherein the usual linked list elements are compressed to occupy roughly half the space), aiding garbage collection by reportedly an order of magnitude. A further improvement was two microcode instructions which specifically supported Lisp functions, reducing the cost of calling a function to as little as 20 clock cycles, in some Symbolics implementations.

The first machine was called the CONS machine (named after the list construction operator cons in Lisp). Often it was affectionately referred to as the Knight machine, perhaps since

Knight wrote his master's thesis on the subject; it was extremely well received. It was subsequently improved into a version called CADR (a pun; in Lisp, the `cadr` function, which returns the second item of a list, is pronounced /'keɪ.dəʃ/ or /'kɑ.dəʃ/, as some pronounce the word "cadre") which was based on essentially the same architecture. About 25 of what were essentially prototype CADRs were sold within and without MIT for ~\$50,000; it quickly became the favorite machine for hacking – many of the most favored software tools were quickly ported to it (e.g. Emacs was ported from ITS in 1975). It was so well received at an AI conference held at MIT in 1978 that Defense Advanced Research Projects Agency (DARPA) began funding its development.

Commercializing MIT Lisp machine technology

In 1979, Russell Noftsker believed that because of the strength of the Lisp language and how it could be further improved with hardware acceleration, Lisp machines presented a commercial opportunity. He proposed commercializing the technology with Greenblatt. Greenblatt, as an AI Lab hacker, was not accustomed to the business environment and hoped to recreate the informal and productive atmosphere of the lab in their venture. His ideas and goals were different from those of Noftsker. The two negotiated at length, but could not find compromise. As the proposed firm could succeed only with the full and undivided assistance of the AI Lab hackers as a group, Noftsker and Greenblatt decided that the fate of the enterprise was up to them, and so the choice should be left to the hackers.

The ensuing discussions of the choice divided the lab into two factions. In February 1979, matters came to a head. The hackers sided with Noftsker, believing that a commercial venture-fund-backed firm had a better chance of surviving and commercializing Lisp machines than Greenblatt's proposed self-sustaining start-up.

It was at this juncture that Symbolics, Noftsker's enterprise, slowly came together. While Noftsker was paying his staff a salary, he had no building or any equipment for the hackers to

work on. He bargained with Patrick Winston that, in exchange for allowing Symbolics' staff to keep working out of MIT, Symbolics would let MIT use internally and freely all the software Symbolics developed. A consultant from CDC, who was trying to put together a natural language computer application with a group of West-coast programmers, came to Greenblatt, seeking a Lisp machine for his group to work with, about eight months after the disastrous conference with Noftsker. Greenblatt had decided to start his own rival Lisp machine firm, but he had done nothing. The consultant, Alexander Jacobson, decided that the only way Greenblatt was going to start the firm and build the Lisp machines that Jacobson desperately needed was if Jacobson pushed and otherwise helped Greenblatt launch the firm. Jacobson pulled together business plans, a board, a partner for Greenblatt (one F. Stephen Wyle). The newfound firm was named LISP Machine, Inc. (LMI), and was funded by CDC orders, via Jacobson.

Around this time Symbolics (Noftsker's firm) began operating. It had been hindered by Noftsker's promise to give Greenblatt a year's head start, and by severe delays in procuring venture capital. Symbolics still had the major advantage that while 3 or 4 of the AI Lab hackers had gone to work for Greenblatt, 14 other hackers had signed onto Symbolics. Two AI Lab people were not hired by either: Richard Stallman and Marvin Minsky. Stallman, however, blamed Symbolics for the decline of the hacker community that had centered around the AI lab. For two years, from 1982 to the end of 1983, Stallman worked by himself to clone the output of the Symbolics programmers, with the aim of preventing them from gaining a monopoly on the lab's computers.

Regardless, after a series of internal battles, Symbolics did get off the ground in 1980/1981, selling the CADR as the LM-2, while Lisp Machines, Inc. sold it as the LMI-CADR. Symbolics did not intend to produce many LM-2s, since the 3600 family of Lisp machines was supposed to ship quickly, but the 3600s were repeatedly delayed, and Symbolics ended up producing ~100 LM-2s, each of which sold for \$70,000. Both firms developed second-generation products

based on the CADR: the Symbolics 3600 and the LMI-LAMBDA (of which LMI managed to sell ~200). The 3600, which shipped a year late, expanded on the CADR by widening the machine word to 36-bits, expanding the address space to 28-bits, and adding hardware to accelerate certain common functions that were implemented in microcode on the CADR. The LMI-LAMBDA, which came out a year after the 3600, in 1983, was compatible with the CADR (it could run CADR microcode), but hardware differences existed. Texas Instruments (TI) joined the fray when it licensed the LMI-LAMBDA design and produced its own variant, the TI Explorer. Some of the LMI-LAMBDA and the TI Explorer were dual systems with both a Lisp and a Unix processor. TI also developed a 32-bit microprocessor version of its Lisp CPU for the TI Explorer. This Lisp chip also was used for the MicroExplorer – a NuBus board for the Apple Macintosh II (NuBus was initially developed at MIT for use in Lisp machines).

Symbolics continued to develop the 3600 family and its operating system, Genera, and produced the Ivory, a VLSI implementation of the Symbolics architecture. Starting in 1987, several machines based on the Ivory processor were developed: boards for Suns and Macs, stand-alone workstations and even embedded systems (I-Machine Custom LSI, 32 bit address, Symbolics XL-400, UX-400, MacIvory II; in 1989 available platforms were Symbolics XL-1200, MacIvory III, UX-1200, Zora, NXP1000 "pizza box"). Texas Instruments shrank the Explorer into silicon as the MicroExplorer which was offered as a card for the Apple Mac II. LMI abandoned the CADR architecture and developed its own K-Machine, but LMI went bankrupt before the machine could be brought to market. Before its demise, LMI was working on a distributed system for the LAMBDA using Moby space.

These machines had hardware support for various primitive Lisp operations (data type testing, CDR coding) and also hardware support for incremental garbage collection. They ran large Lisp programs very efficiently. The Symbolics machine was competitive against many commercial super minicomputers, but was never adapted for conventional purposes. The Symbolics Lisp Machines were also sold to some non-AI markets like computer graphics,

modeling, and animation.

The MIT-derived Lisp machines ran a Lisp dialect named Lisp Machine Lisp, descended from MIT's Maclisp. The operating systems were written from the ground up in Lisp, often using object-oriented extensions. Later, these Lisp machines also supported various versions of Common Lisp (with Flavors, New Flavors, and Common Lisp Object System (CLOS)).

Interlisp, BBN, and Xerox

Bolt, Beranek and Newman (BBN) developed its own Lisp machine, named Jericho, which ran a version of Interlisp. It was never marketed. Frustrated, the whole AI group resigned, and were hired mostly by Xerox. So, Xerox Palo Alto Research Center had, simultaneously with Greenblatt's own development at MIT, developed their own Lisp machines which were designed to run InterLisp (and later Common Lisp). The same hardware was used with different software also as Smalltalk machines and as the Xerox Star office system. These included the Xerox 1100, Dolphin (1979); the Xerox 1132, Dorado; the Xerox 1108, Dandelion (1981); the Xerox 1109, Dandetiger; and the Xerox 1186/6085, Daybreak. The operating system of the Xerox Lisp machines has also been ported to a virtual machine and is available for several platforms as a product named Medley. The Xerox machine was well known for its advanced development environment (InterLisp-D), the ROOMS window manager, for its early graphical user interface and for novel applications like NoteCards (one of the first hypertext applications).

Xerox also worked on a Lisp machine based on reduced instruction set computing (RISC), using the 'Xerox Common Lisp Processor' and planned to bring it to market by 1987, which did not occur.

Integrated Inference Machines

In the mid-1980s, Integrated Inference Machines (IIM) built prototypes of Lisp machines

named Inferstar.

Developments of Lisp machines outside the United States

In 1984–85 a UK firm, Racal-Norsk, a joint subsidiary of Racal and Norsk Data, attempted to repurpose Norsk Data's ND-500 supermini as a microcoded Lisp machine, running CADR software: the Knowledge Processing System (KPS).

There were several attempts by Japanese manufacturers to enter the Lisp machine market: the Fujitsu Facom-alpha mainframe co-processor, NTT's Elis, Toshiba's AI processor (AIP) and NEC's LIME. Several university research efforts produced working prototypes, among them are Kobe University's TAKITAC-7, RIKEN's FLATS, and Osaka University's EVLIS.

In France, two Lisp Machine projects arose: M3L at Toulouse Paul Sabatier University and later MAIA.

In Germany Siemens designed the RISC-based Lisp co-processor COLIBRI.

End of the Lisp machines

With the onset of an AI winter and the early beginnings of the microcomputer revolution, which would sweep away the minicomputer and workstation makers, cheaper desktop PCs soon could run Lisp programs even faster than Lisp machines, with no use of special-purpose hardware. With their high profit margin hardware business eliminated, most Lisp machine makers had gone out of business by the early 90s, leaving only software-based firms like Lucid Inc. or hardware makers who had switched to software and services to avoid the crash. As of January 2015, besides Xerox and TI, Symbolics is the only Lisp machine firm still operating, selling the Open Genera Lisp machine software environment and the Macsyma computer algebra system.

Legacy

Several attempts to write open-source emulators for various Lisp Machines have been made: CADR Emulation, Symbolics L Lisp Machine Emulation, the E3 Project (TI Explorer II Emulation), Meroko (TI Explorer I), and Nevermore (TI Explorer I). On 3 October 2005, the MIT released the CADR Lisp Machine source code as open source.

In September 2014, Alexander Burger, developer of PicoLisp, announced PiMCU, an implementation of PicoLisp in hardware.

The Bitsavers' PDF Document Archive has PDF versions of the extensive documentation for the Symbolics Lisp Machines, the TI Explorer and MicroExplorer Lisp Machines and the Xerox Interlisp-D Lisp Machines.

Applications

Domains using the Lisp machines were mostly in the wide field of artificial intelligence applications, but also in computer graphics, medical image processing, and many others.

The main commercial expert systems of the 80s were available: Intellicorp's Knowledge Engineering Environment (KEE), Knowledge Craft, from The Carnegie Group Inc., and ART (Automated Reasoning Tool) from Inference Corporation.

Technical overview

Initially the Lisp machines were designed as personal workstations for software development in Lisp. They were used by one person and offered no multi-user mode. The machines provided a large, black and white, bitmap display, keyboard and mouse, network adapter, local hard disks, more than 1 MB RAM, serial interfaces, and a local bus for extension cards. Color graphics cards, tape drives, and laser printers were optional.

The processor did not run Lisp directly, but was a stack machine with instructions optimized for compiled Lisp. The early Lisp machines used microcode to provide the instruction set. For several operations, type checking and dispatching was done in hardware at runtime. For

example, one addition operation could be used with various numeric types (integer, float, rational, and complex numbers). The result was a very compact compiled representation of Lisp code.

The following example uses a function that counts the number of elements of a list for which a predicate returns true.

The disassembled machine code for above function (for the Ivory microprocessor from Symbolics):

The operating system used virtual memory to provide a large address space. Memory management was done with garbage collection. All code shared a single address space. All data objects were stored with a tag in memory, so that the type could be determined at runtime. Multiple execution threads were supported and termed processes. All processes ran in the one address space.

All operating system software was written in Lisp. Xerox used Interlisp. Symbolics, LMI, and TI used Lisp Machine Lisp (descendant of MacLisp). With the appearance of Common Lisp, Common Lisp was supported on the Lisp Machines and some system software was ported to Common Lisp or later written in Common Lisp.

Some later Lisp machines (like the TI MicroExplorer, the Symbolics MacIvory or the Symbolics UX400/1200) were no longer complete workstations, but boards designed to be embedded in host computers: Apple Macintosh II and Sun-3 or Sun-4.

Some Lisp machines, such as the Symbolics XL1200, had extensive graphics abilities using special graphics boards. These machines were used in domains like medical image processing, 3D animation, and CAD.

See also

ICAD - example of knowledge-based engineering software originally developed on a Lisp machine, later ported to other platforms

Orphaned technology

References

General

External links

Symbolics website

Medley

Bitsavers, PDF documents

LMI documentation

MIT CONS documentation

MIT CADR documentation

Lisp Machine Manual, Chinual

"The Lisp Machine manual, 4th Edition, July 1981"

"The Lisp Machine manual, 6th Edition, HTML/XSL version"

"The Lisp Machine manual"

Information and code for LMI Lambda and LMI K-Machine

Jaap Weel's Lisp Machine Webpage at the Wayback Machine (archived 23 June 2015) – A set of links and locally stored documents regarding all manner of Lisp machines

"A Few Things I Know About LISP Machines" – A set of links, mostly discussion of buying Lisp machines

Ralf Möller's Symbolics Lisp Machine Museum

Vintage Computer Festival pictures of some Lisp machines, one running Genera

LISPMachine.net – Lisp Books and Information

Lisp machines timeline – a timeline of Symbolics' and others' Lisp machines

(in French) "Présentation Générale du projet M3L" – An account of French efforts in the same vein

Discussion

"If It Works, It's Not AI: A Commercial Look at Artificial Intelligence startups"

"Symbolics, Inc.: A failure of Heterogenous engineering" – (PDF)

"My Lisp Experiences and the Development of GNU Emacs" – transcript of a speech Richard Stallman gave about Emacs, Lisp, and Lisp machines